

COMP0008: Computer Architecture & Concurrency – CW2 Solution

Part 1: Instruction encoding:

Without using MARS:

1. write the 32-bit binary MIPS representation of mult \$13,\$15.

Solution:

Looking at the MIPS card, we can see the following spec:

Multiply mult R {Hi,Lo} = R[rs] * R[rt] 0/--/--/18

This means that the opcode is 0 (as in all R-type instructions) and the function code is (hex!) 18.

Recall that the format for R-type instructions (see week 2's lecture) is:



We see that the current instruction doesn't use rd (so by default it's 0), and it's not a shift instruction (so shamt is also 0), so overall we get:



So, we got the encoding 0b000000011010111100000000000011000 = 0x1af0018.

2. Disassemble and understand the following binary code:

0x210affff

0x010a4824

0x0009582b

- Write the three instructions in assembly.

Solution:

Let us first convert the instructions to binary and highlight the opcode:

0x210affff = 0b00100001000010101111111111111111

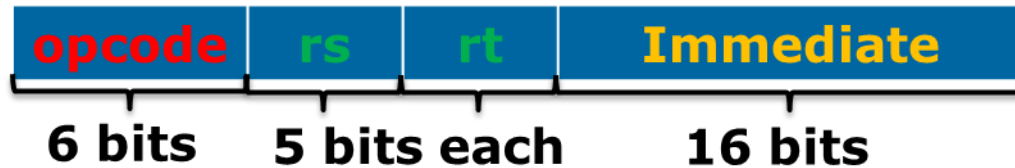
0x010a4824 = 0b00000001000010100100100000100100

0x0009582b = 0b000000000000010010101100000101011

So, we see that the opcodes are 8, 0, and 0.

Looking at the reference card, we see that opcode=8 is the instruction addi, which is an I-type instruction.

This means that the format is (week 3):



This means that we split the encoding as 0b001000010000101011111111111111.

That gives rs = 8, rt = 10 and Imm = 0xffff = -1 (as a two's complement number).

Thus, the instruction is addi \$10, \$8, -1. (Notice that rt comes first!)

The next two instructions have opcode=0, which suggest that they are R-type and we need to look at their function code, highlighted in teal:

0x010a4824 = 0b00000001000010100100100000100100

0x0009582b = 0b000000000000010010101100000101011

Hence, the first of these have a function code of 0x24 while the second has 0x2B.

A quick peek at the reference card gives:

And and R R[rd] = R[rs] & R[rt] 0 / 24_{hex}
Set Less Than Unsig. sltu R R[rd] = (R[rs] < R[rt]) ? 1 : 0 (6) 0 / 2b_{hex}

Next, we need to extract the rs/rt/rd register numbers:

0x010a4824 = 0b00000001000010100100100000100100

0x0009582b = 0b000000000000010010101100000101011

This means that the instructions are: and \$9, \$8, -1 and sltu \$11, \$0, \$9.

- **(Challenging!)** For which values of \$8, is \$11 going to be 1 after these instructions are executed?

Note: One of these instructions is an I-type instruction, whose format will be discussed in week 3's lecture. Try inferring the structure from the MIPS reference card!

Solution:

Let us take a close look at the combination of these instructions:

```
addi $10, $8, -1
and $9, $8, $10
sltu $11, $0, $9
```

The first one sets \$10 to $\$8 - 1$ and the second sets \$9 to the bitwise-and of \$8 and \$10. That is, \$9 would get the bitwise and of \$8 and $(\$8 - 1)$.

The third instruction sets \$11 is 1 if \$0 (which, as you recall, always zero) is less than or equal to \$9, in an unsigned comparison.

As an unsigned comparison, 0 is the smallest number, so the question is whether $\$9 = 0$.

Putting it together, we get that \$11 will be one when $\$9 = \$8 \& (\$8 - 1)$ is **non-zero**.

Let us take an example. Suppose that $\$8 = 0b1010$. Then $\$8 - 1 = 0b1001$ and $\$8 \& (\$8 - 1) = 0b1000$.

Another example. Say that $\$8 = 0b1000$. Thus, $\$8 - 1 = 0b111$ and $\$8 \& (\$8 - 1) = 0$.

This seems to suggest that $\$8 \& (\$8 - 1) = 0$ if and only if \$8 is a power of two. This suggestion is correct; think of how you'd prove it (**hint**: take a look at the most significant 1 bit).

To conclude, the three instructions test whether \$8 is a power of two. If it is, then they set $\$11 = 0$ and otherwise $\$11 = 1$.

Before we move to parts 2 and 3, a bit of context:

In this CW, I did not explicitly state what are the metrics on which your code will be judged, since the CW is not marked.

If I'd want you to write a code in a specific way in the exam (e.g., an efficient code or a code that doesn't modify specific registers), I will be super explicit about that.

To get a feeling of what I mean by that, my answer here for part 2 simply provides a "correct code", while for part 3 I give solutions that are efficient and also respect the register convention (specifically, I show how to implement the pseudo instructions without modifying any unwanted registers).

Part 2: The Collatz conjecture:

The [Collatz conjecture](#) is that any Collatz sequence, defined by an input integer a_0 , always reaches the value 1 after a finite number of steps.

Starting with the number a_0 , the sequence is defined as
$$a_{n+1} = \begin{cases} 3a_n + 1 & \text{if } (a_n \bmod 2) = 1 \\ a_n/2 & \text{otherwise} \end{cases} .$$

For example, if $a_0 = 7$, then the sequence is 7,22,11,34,17,52,26,13,40,20,10,5,16,8,4,2,1,4,2,1, ...

It was recently ([July 2021](#)) announced that whoever solves the problem would win a prize larger than \$1M.

In this CW, we will (probably) not solve the conjecture.

Instead, you'll be writing a MIPS32 assembly program that will print the sequence (for up to 500 elements) of the sequence, given a user input a_0 .

Specifically, write a code that takes an integer as input and prints the sequence numbers until the first 1.

You may assume that none of the numbers exceed $2^{31} - 1$ and can thus be represented using 32-bit integers.

Remember: when programming in MIPS, the reference card ([link on the website](#)) is your friend!

Solution:

*Remember: there are **many** ways of writing a code that does the same task. It's better to try it yourself first! The code below is an example solution and is not optimized in any way.*

```
# Compute up to 500 number following the input number in its Collatz sequence

.data
output: .word 0 : 500      # "array" of 500 values for the output
size:   .word 500          # size of "array"

.text

la $t0, output            # load address of output array
la $t5, size               # load address of size variable
lw $t5, 0($t5)            # load array size
li $v0, 5                 # MARS call number 5: read int
syscall                   # return value in $v0
move $t1, $v0             # store initial value in t1
li $t6, 1                 # $t6 = 1
li $t7, 3                 # $t7 = 3
sw $t1, 0($t0)            # a[0] = input number
addi $t1, $t5, -1         # Counter for loop, will execute (size-2) times

loop:
lw $t3, 0($t0)            # Get value from array a[n] into $t3
andi $t2, $t3, 1         # $t2 = t3 % 2
beq $t2, $t6, odd         # if (t2 == 1) , i.e., if a[n] was an odd number #even:
srl $t2, $t3, 1           # $t2 = $t3 / 2 = a[n] / 2
j updated

odd:
multu $t3, $t7            # compute t3 * 3 = a[n] * 3
mflo $t3                  # move the result to $t3, i.e., $t3 = a[n] * 3
addi $t2, $t3, 1          # Set $t2 = $t3 + 1 = a[n] * 3 + 1
```

```

updated:  # Done computing ($t2 = a[n] / 2) or ($t2 = a[n] * 3 + 1) depending on whether a[n] was even

    sw    $t2, 4($t0)        # Store a[n+1] in array
    addi  $t0, $t0, 4        # increment address
    addi  $t1, $t1, -1       # decrement loop counter
    beq   $t2, $t6, done
    bgtz  $t1, loop          # repeat if not finished yet.

done:

    la    $a0, output        # first argument for print (array)
    subu  $a1, $t5, $t1      # compute the output size
    jal   print              # call print routine.
    li    $v0, 10            # system call for exit
    syscall                                # we are out of here.

##### routine to print the numbers on one line.

    .data
space:.asciiz  " "           # space to insert between numbers
head: .asciiz  "The Collatz sequence is:\n"

    .text

print:

    add   $t0, $zero, $a0    # starting address of array
    add   $t1, $zero, $a1    # initialize loop counter to array size
    la    $a0, head          # load address of print heading
    li    $v0, 4              # specify Print String service
    syscall                                # print heading

out:

    lw    $a0, 0($t0)        # load next Collatz number for syscall
    li    $v0, 1             # specify Print Integer service
    syscall                                # print Collatz number
    la    $a0, space         # load address of spacer for syscall
    li    $v0, 4              # specify Print String service
    syscall                                # output string
    addi  $t0, $t0, 4        # increment address
    addi  $t1, $t1, -1       # decrement loop counter
    bgtz  $t1, out           # repeat if not finished
    jr    $ra                # return

```

Part 3: Implementing complex pseudo instructions

MARS, which you will be using in this CW, has an assembler that replaces pseudo-instructions with (one or more) hardware supported instructions.

On a different topic, out-of-order-execution (OOOE) CPUs (more on that later in the module!) suffer great performance hit when a program contains many branches.

This means that branch-less programming is often essential for writing performant code.

In this assignment, you'll design an implementation of two new "pseudo instructions" that are not currently supported on MARS:

- `min $t0, $t1, $t2` (set `$t0 = min($t1,$t2)`).
- `med $t0, $t1, $t2` (set `$t0` to the median of `$t0,$t1,$t2`).

As this is not the focus of the module, we will not actually modify MARS's code. Instead you will:

3. Write a code that prints out the minimum of two registers `$t1,$t2` (which will be set at the beginning of the code). Your code must be branchless!
4. **(Challenging!)** Write a code that prints out the median of three registers `$t0,$t1,$t2` (which will be set at the beginning of the code). Your code must be branchless!

Solution:

Off the bat, I want to emphasize that this was not required in the current exercise, but pseudo-instructions have only one scratch register: \$1 (a.k.a. \$at).

This means that, if we were to actually implement these as part of an assembler, we would need to save any other register that we modify and restore its value before finishing.

Again, there are many possible solutions for the task. The easiest of which is to use the slt instruction followed by mult, for example:

```
### Actual branchless code of computing $t0 <- min($t1, $t2)

slt $1, $t1, $t2
mult $1, $t1
mflo $1      #$1 gets $t1 if ($t1 < $t2)
slt $t0, $t2, $t1
mult $t0, $t2
mflo $t0     #$t0 gets $t2 if ($t2 < $t1)
add $t0, $t0, $1
###
```

*While this code seems intuitive and easy – **it is incorrect!** (Think of what happens for $t1 = t2$).*

A correct (although weird looking!) code can be:

```
### Actual branchless code of computing $t0 <- min($t1, $t2)

slt $1, $t1, $t2                # $1 = 1 if ($t1 < $t2)
sub $t0, $t1, $t2
mult $1, $t0
mflo $1                          # $1 = ($t1-$t2) if ($t1 < $t2) else 0
add $t0, $0, $t2                # $t0 = $t2
add $t0, $t0, $1                # $t0 = $t2 + $1 = min($t1, $t2)
###
```

As it turns out, we can get a more efficient code by using bitwise instructions!

For example, consider the following:

```
### Actual branchless code of computing $t0 <- min($t1, $t2)
subu $1, $t1, $t2

sra $t0, $1, 31      # $t0 = 0xffffffff if ($t1 < $t2) and 0 otherwise
and $t0, $t0, $1      # $t0 = ($t1-$t2) if ($t1 < $t2) and 0 otherwise
add $t0, $t0, $t2

###
```

Keep this in mind (for the exam, and in general if you program) – bitwise operations can very often simplify your code and make it faster!

In fact, we could even do it without \$1, at the cost of an extra instruction:

```
### Actual branchless code of computing $t0 <- min($t1, $t2)
subu $t1, $t1, $t2    # Don't worry, we'll correct that!
sra $t0, $t1, 31      # $t0 = 0xffffffff if ($t1 < $t2) and 0 otherwise
and $t0, $t0, $t1      # $t0 = ($t1-$t2) if ($t1 < $t2) and 0 otherwise
addu $t0, $t0, $t2
addu $t1, $t1, $t2     # restore $t1 to its original value

###
```

While avoiding the use of scratch registers is generally unnecessary, complex functions or pseudo instructions may require more scratch registers, so using them wisely is important.

The second exercise asked us to compute the median.

Again, there are different methods of doing so.

One option is to leverage our min code for that, e.g., using the following equation:

$$\text{med}(\$t0, \$t1, \$t2) = \max(\min(\$t0, \$t1), \min(\max(\$t0, \$t1), \$t2)).$$

Computing the max is similar to min, so that is one way to go.

But why not use some more bitwise 😊?

The crux of our efficient median computation is this:

1. *if exactly one of the following holds, then \$t1 is the median:*
 - a. $\$t1 > \$t0$
 - b. $\$t1 > \$t2$
2. *If exactly one of the following holds, then \$t2 is the median:*
 - a. $\$t1 > \$t2$
 - b. $\$t0 > \$t2$
3. *If (1) and (2) didn't hold, \$t0 is the median and we're done.*

*How do we turn this into code? The notion of “exactly one out of two” is simply **xor**!*

*How do we check both (1) and (2) didn't hold? That is **nor**!*

Note: *we will start by using multiple additional registers, see below on how to get away with using only \$1!*

Actual branchless code of computing $\$t0 \leftarrow \text{med}(\$t0, \$t1, \$t2)$

```
subu $t3, $t1, $t0    # if ($t3<1) then ($1<$0)
subu $t4, $t1, $t2    # if ($t4<1) then ($1<$2)
xor $t3, $t3, $t4      # if ($t3<0) then $t1 is the median by rule (1)
sra $t3, $t3, 31       # if $t1 is the median, then $t3=0xFFFFFFFF else 0
```

```
subu $t5, $t0, $t2
xor $t5, $t5, $t4      # if ($t5<0) then $t2 is the median by rule (2)
sra $t5, $t5, 31       # if $t2 is the median, then $t5=0xFFFFFFFF else 0
```

```
nor $t6, $t5, $t3      # if $t0 is the median (rule (3)), then $t6=0xFFFFFFFF else 0
and $t5, $t5, $t2      # if $t2 is the median, then $t5=$t2 else 0
and $t3, $t3, $t1      # if $t1 is the median, then $t3=$t1 else 0
and $t0, $t0, $t6      # if $t0 is the not median, set $t0=0
addu $t0, $t0, $t3
addu $t0, $t0, $t5
###
```

Okay, so we have our first solution, which is actually quite efficient. But can we improve it further?

We can, with the help of the instruction (which we haven't seen, but it's supported) `movn`.

`movn` is a **conditional move** instruction.

Namely, `movn $t4, $t5, $t6` would set `$t4` to `$t5` **only if** `$t6` $\neq 0$.

(Hence the name **move non-zero**).

The instruction does appear on the reference card (but not documented otherwise). It is documented, however, in the textbook.

OPCODES, BASE CONVERSION, ASCII SYMBOLS

MIPS opcode (31:26)	(1) MIPS funct (5:0)	(2) MIPS funct (5:0)	Binary	Deci- mal	Hexa- deci- mal	ASCII Char- acter
(1)	sll	add, <i>f</i>	00 0000	0	0	NUL
		sub, <i>f</i>	00 0001	1	1	SOH
j	srl	mul, <i>f</i>	00 0010	2	2	STX
jal	sra	div, <i>f</i>	00 0011	3	3	ETX
beq	sllv	sqr, <i>f</i>	00 0100	4	4	EOT
bne		abs, <i>f</i>	00 0101	5	5	ENQ
blez	srlv	mov, <i>f</i>	00 0110	6	6	ACK
bgtz	srav	neg, <i>f</i>	00 0111	7	7	BEL
addi	jr		00 1000	8	8	BS
addiu	jalr		00 1001	9	9	HT
slti	movz		00 1010	10	a	LF
sltiu	<code>movn</code>		00 1011	11	b	VT

Let us see how we can use it:

Actual branchless code of computing `$t0 <- med($t0, $t1, $t2)`

```
slt $1, $t0, $t1
```

```
slt $t4, $t2, $t1
```

```
xor $1, $1, $t4      #$1 = ($t0 < $t1) ^ ($t2 < $t1)
```

```
movn $t0, $t1, $1     #$t0 = $t1 if ($t0 < $t1) ^ ($t2 < $t1), otherwise it remains the input $t0
```

```
slt $1, $t2, $t0
```

```
xor $1, $1, $t4      #$1 = ($t2 < $t0) ^ ($t2 < $t1)
```

```
movn $t0, $t2, $1     #$t0 = $t1 if ($t2 < $t0) ^ ($t2 < $t1), otherwise it remains the input $t0
```

###

Not bad, we got down to 7 instructions!

*But in practice (**not** for the purpose of this exercise!) such a code is unacceptable, since the assembler is not allowed to modify \$t4 for implementing pseudo instructions.*

What can we do? With some additional tricks (and instructions) we can avoid using any additional register:

Actual branchless code of computing $\$t0 \leftarrow \text{med}(\$t0, \$t1, \$t2)$

```
subu $1, $t0, $t1      #if ($1 < 0) then ($t0 < $t1)
subu $t2, $t2, $t1      # $t2 = $t2 - $t1, remember to fix it!
xor $1, $1, $t2         #If ($1 < 0) then ($t0 < $t1) ^ ($t0 < $t2)
sra $1, $1, 31          #If ($1 ≠ 0) then ($t0 < $t1) ^ ($t0 < $t2)

movn $t0, $t1, $1       # $t1 is the median!

subu $1, $t2, $t0        #we want $1 = $t2 - $t0, but $t2 holds $t2 - $t1, so we got $1 = ($t2 - $t1) - $t0
addu $1, $1, $t1         #fix $1 to hold $t2 - $t0
xor $1, $1, $t2         # $1 = ($t2 - $t0) ^ ($t2 - $t1), so it's negative only if $t2 is the median.
sra $1, $1, 31
add $t2, $t2, $t1       # $t2 is restored
movn $t0, $t2, $1

###
```

(Note: several of the code pieces above assume that no overflow happens. In case \$t0-\$t2 may be very large/small, we can fix this using some additional logic.)